OS Unit III - CPU Scheduling

Unit III – CPU Scheduling

Operating System | MSBTE K-Scheme | Diploma CO Branch | TY

3.1 Scheduling

Basic Concept

In a multiprogramming system, several processes reside in memory at the same time. When one process has to wait (for example, for an I/O operation), the OS takes the CPU away from it and gives it to another process. This way, the CPU is never left idle — this is the basic idea behind **CPU scheduling**.

CPU scheduling is the process of determining which process in the ready queue is allocated the CPU next. It is the basis of multiprogrammed operating systems and helps maximize CPU utilization.

Goal: Whenever the CPU becomes idle, the OS must select one of the processes from the ready queue to be executed, using a **CPU scheduler (short-term scheduler)**.

CPU and I/O Burst Cycle

Process execution consists of a cycle of **CPU execution** and **I/O wait**:

1. A process starts with a **CPU burst** (a period of CPU execution).
2. It is then followed by an **I/O burst** (a period of waiting for I/O).
3. This continues, alternating between CPU bursts and I/O bursts, until the process finishes with a final CPU burst that ends execution.

Diagram (described):

```
Process:  [CPU burst] -> [I/O burst] -> [CPU burst] -> [I/O burst] -> ... -> [CPU burst] -> ends
```

Types of processes based on bursts: - **CPU-bound process** – has long CPU bursts and fewer, shorter I/O bursts (e.g., mathematical computation programs). - **I/O-bound process** – has short CPU bursts and frequent, longer I/O bursts (e.g., programs that do a lot of file/keyboard operations).

The **CPU burst distribution** of processes is important for choosing an appropriate scheduling algorithm — most processes follow a pattern of many short CPU bursts and few long ones.

3.2 Preemptive and Non-Preemptive Scheduling, Scheduling Criteria

Preemptive Scheduling

- The CPU can be **taken away** from a running process before it completes its burst, if a higher-priority process arrives or a time slice expires.
- Used when quick response is important (interactive/time-sharing systems).
- Examples: Round Robin, Shortest Remaining Time Next (SRTN), Priority Scheduling (preemptive version).
- **Disadvantage:** Involves overhead of context switching.

Non-Preemptive Scheduling

- Once the CPU is allocated to a process, it **keeps the CPU until it terminates or switches to the waiting state** (voluntarily releases the CPU) — the OS cannot forcibly remove it.
- Simpler to implement, but a long process can make short processes wait a long time (poor response time).
- Examples: First Come First Serve (FCFS), Shortest Job First (SJF, non-preemptive version).

Comparison Table: | Preemptive | Non-Preemptive | |—|—| | CPU can be taken away from a process | CPU cannot be taken away until process finishes/waits | | Higher overhead (context switching) | Lower overhead | | Better response time | Can cause longer waiting time for short jobs | | Used in time-sharing/real-time systems | Used in batch systems |

Scheduling Criteria

Different CPU scheduling algorithms have different properties. The following criteria are used to compare them:

1. **CPU Utilization** – keep the CPU as busy as possible (ideally close to 100%).
2. **Throughput** – number of processes completed per unit time.
3. **Turnaround Time** – total time taken from submission of a process to its completion. `Turnaround Time = Completion Time − Arrival Time`
4. **Waiting Time** – total time a process spends waiting in the ready queue. `Waiting Time = Turnaround Time − Burst Time`
5. **Response Time** – time from submission of a request until the first response is produced (important for interactive systems), not the time until output is complete.

General goal: Maximize CPU utilization and throughput; minimize turnaround time, waiting time, and response time.

3.3 Types of Scheduling Algorithms

3.3.1 First Come First Serve (FCFS)

- The process that requests the CPU **first** is allocated the CPU first.
- Implemented using a **FIFO queue**.
- **Non-preemptive** in nature.

Working: When a process enters the ready queue, its PCB is linked to the tail of the queue. When the CPU is free, it is allocated to the process at the head of the queue.

Example: | Process | Burst Time | |—|—| | P1 | 24 | | P2 | 3 | | P3 | 3 |

If processes arrive in order P1, P2, P3: - Gantt Chart: `| P1 | P2 | P3 |` → P1: 0–24, P2: 24–27, P3: 27–30 - Waiting time: P1 = 0, P2 = 24, P3 = 27 → Average = (0+24+27)/3 = 17

Drawback – Convoy Effect: If a long process arrives first, all shorter processes behind it must wait a long time, reducing CPU and device utilization. This is called the **convoy effect**.

3.3.2 Shortest Job First (SJF)

- The process with the **smallest CPU burst time** is selected next for execution.
- Can be **non-preemptive** (once started, runs to completion) or **preemptive** (see SRTN below).
- Also called **Shortest Job Next (SJN)**.

Advantage: Gives the minimum average waiting time for a given set of processes (provably optimal for non-preemptive scheduling).

Disadvantage: - Difficult to know the exact length of the next CPU burst in advance (usually predicted using past burst history). - May cause **starvation** of longer processes if shorter processes keep arriving.

Example: | Process | Burst Time | |—|—| | P1 | 6 | | P2 | 8 | | P3 | 7 | | P4 | 3 |

Order of execution (shortest first): P4(3) → P1(6) → P3(7) → P2(8) - Gantt Chart: `| P4 | P1 | P3 | P2 |`

3.3.3 Shortest Remaining Time Next (SRTN)

- This is the **preemptive version of SJF**.
- If a new process arrives with a CPU burst length **shorter than the remaining time** of the currently executing process, the CPU is preempted (taken away) and given to the new process.
- Also called **Shortest Remaining Time First (SRTF)**.

Key Point: Requires continuous comparison of remaining burst times of all processes in the ready queue, so it has more overhead than non-preemptive SJF, but it minimizes average waiting time even further when process arrival times differ.

Example: If P1 (burst 8) starts running, and after 1 unit P2 arrives with burst 4, since 4 < 7 (remaining time of P1), the CPU is switched to P2 immediately.

3.3.4 Round Robin (RR)

- Designed especially for **time-sharing systems**.
- Similar to FCFS, but **preemptive** — each process gets a small unit of CPU time called a **time quantum (time slice)**, usually 10–100 milliseconds.
- After a process's time quantum expires, it is preempted and added to the **end of the ready queue**; the CPU moves to the next process.

Working: - The ready queue is treated as a **circular queue**. - The scheduler picks the first process from the queue, sets a timer to interrupt after 1 time quantum, and dispatches the process. - If the process's burst is longer than the quantum, it is preempted and placed at the back of the queue; if shorter, the process releases the CPU voluntarily when it finishes/blocks.

Performance depends heavily on the size of the time quantum: - If the quantum is **too large** → RR behaves like FCFS. - If the quantum is **too small** → too many context switches occur, increasing overhead. - A good rule of thumb: 80% of CPU bursts should be shorter than the time quantum.

Example: Time quantum = 4 | Process | Burst Time | |—|—| | P1 | 24 | | P2 | 3 | | P3 | 3 |

Gantt Chart: `| P1 | P2 | P3 | P1 |` (P1 runs 0–4, P2 runs 4–7 completes, P3 runs 7–10 completes, P1 continues 10–24 to finish)

3.3.5 Priority Scheduling

- A **priority number (integer)** is associated with each process.
- The CPU is allocated to the process with the **highest priority** (commonly, the smallest integer represents the highest priority).
- Can be **preemptive** (a newly arrived higher-priority process preempts the running process) or **non-preemptive** (new higher-priority process just goes to the head of the ready queue).

SJF is a special case of priority scheduling, where the priority is the inverse of the predicted next CPU burst.

Problem – Starvation (Indefinite Blocking): Low-priority processes may never execute if high-priority processes keep arriving continuously.

Solution – Aging: Gradually **increase the priority** of processes that wait in the system for a long time, ensuring they eventually get executed.

3.3.6 Multilevel Queue Scheduling

- Used when processes can be easily classified into different groups based on their nature — e.g., **foreground (interactive)** processes and **background (batch)** processes.
- The **ready queue is divided into several separate queues**, and each process is permanently assigned to one queue based on a property (memory size, process type, priority, etc.).
- **Each queue has its own scheduling algorithm** (e.g., foreground queue may use Round Robin, background queue may use FCFS).

Scheduling between queues: 1. **Fixed priority scheduling** – each queue has absolute priority over lower-priority queues (e.g., no process in the batch queue runs unless all foreground queues are empty). Can cause starvation of low-priority queues. 2. **Time slicing between queues** – each queue gets a certain portion of CPU time, which it schedules among its own processes (e.g., 80% CPU time to foreground queue using RR, 20% to background queue using FCFS).

Diagram (described):

```
Highest Priority --> [System Processes Queue]
                 --> [Interactive Processes Queue]
                 --> [Interactive Editing Queue]
                 --> [Batch Processes Queue]
Lowest Priority  --> [Student Processes Queue]
```

Each queue is scheduled independently, and queues are also scheduled relative to each other.

3.4 Deadlock

What is a Deadlock?

A **deadlock** is a situation in which two or more processes are unable to proceed because each is waiting for a resource held by another process in the same set — none of them can proceed, and this waiting continues forever unless the OS intervenes.

Example: Process A holds Resource 1 and requests Resource 2; Process B holds Resource 2 and requests Resource 1 → both wait forever.

3.4.1 System Model

A computer system consists of a finite number of **resources** to be distributed among a number of competing processes. Resources are partitioned into several types (CPU cycles, memory space, I/O devices, files), each with a number of identical instances.

A process must follow this sequence to use a resource: 1. **Request** – the process requests the resource. If not available immediately, the requesting process must wait. 2. **Use** – the process operates/uses the resource. 3. **Release** – the process releases the resource after use.

This request-use-release sequence, when combined across multiple processes competing for limited resources, can lead to deadlock.

Resource Allocation Graph (RAG): Deadlocks can be modeled using a directed graph: - **Process nodes** (circles) and **Resource nodes** (rectangles). - A **request edge** (Process → Resource) shows a process requesting a resource. - An **assignment edge** (Resource → Process) shows a resource assigned to a process. - If the graph contains a **cycle**, a deadlock **may** exist (definitely exists if each resource type has only one instance).

3.4.2 Necessary Conditions Leading to Deadlock

A deadlock can arise **only if all four of the following conditions hold simultaneously**:

1. **Mutual Exclusion** – At least one resource must be held in a non-shareable mode; only one process can use the resource at a time.

2. **Hold and Wait** – A process holding at least one resource is waiting to acquire additional resources that are currently held by other processes.
3. **No Preemption** – A resource can only be released voluntarily by the process holding it, after it has completed its task; it cannot be forcibly taken away.
4. **Circular Wait** – A set of waiting processes {P0, P1, …, Pn} exists such that P0 is waiting for a resource held by P1, P1 is waiting for a resource held by P2, …, and Pn is waiting for a resource held by P0 — forming a cycle.

Note: All four conditions must hold together for a deadlock to occur; preventing even one of them prevents deadlock.

3.4.3 Deadlock Handling Methods

There are three general approaches to deal with deadlocks: 1. **Deadlock Prevention** – ensure the system never enters a deadlock state by design. 2. **Deadlock Avoidance** – allow all conditions but make careful resource allocation decisions to avoid unsafe states. 3. **Deadlock Detection and Recovery** – allow the system to enter a deadlock, then detect and recover from it.

Deadlock Prevention

Prevention works by ensuring that **at least one of the four necessary conditions cannot hold**:

1. **Denying Mutual Exclusion** – Not always possible, since some resources (like printers) are inherently non-shareable. Where possible, use shareable resources (e.g., read-only files).
2. **Denying Hold and Wait** – Require processes to request and be allocated **all resources at once**, before execution begins; or require a process to release all currently held resources before requesting new ones.
    - Drawback: Low resource utilization, possible starvation.
3. **Denying No Preemption** – If a process holding resources requests another resource that cannot be immediately allocated, all resources currently held are **preempted** (released). The process is restarted later only when it can regain its old resources plus the new ones.
4. **Denying Circular Wait** – Impose a **total ordering** of all resource types, and require that each process requests resources in an **increasing order of enumeration**. This way, a cycle cannot form.

Deadlock Avoidance – Banker's Algorithm

Deadlock avoidance requires that the system have **advance knowledge** of the maximum resources a process may request. The system decides, for each resource request, whether allocating it would leave the system in a **safe state**.

Safe State: A state is safe if the system can allocate resources to each process (up to its maximum) in **some order** and still avoid deadlock — i.e., there exists a safe sequence of all processes.

Banker's Algorithm — Named because it is analogous to how a bank ensures it never allocates its cash in a way that it can't satisfy the needs of all its customers.

Data structures used (let n = number of processes, m = number of resource types): - **Available** [m] – number of available instances of each resource type. - **Max** [n][m] – maximum demand of each process for each resource type. - **Allocation** [n][m] – number of resources of each type currently allocated to each process. - **Need** [n][m] – remaining resource need of each process. `Need[i][j] = Max[i][j] − Allocation[i][j]`

Safety Algorithm (to check if a state is safe): 1. Let `Work = Available` and `Finish[i] = false` for all i. 2. Find a process i such that `Finish[i] = false` and `Need[i] ≤ Work`. If no such process exists, go to step 4. 3. Set `Work = Work + Allocation[i]` and `Finish[i] = true`. Go back to step 2. 4. If `Finish[i] = true` for all i, the system is in a **safe state**.

Resource Request Algorithm (when process Pi requests resources, Request[i]): 1. If `Request[i] ≤ Need[i]`, go to step 2; otherwise, raise an error (process exceeded its maximum claim). 2. If `Request[i] ≤`

`Available`, go to step 3; otherwise, Pi must wait (resources not available). 3. **Pretend** to allocate the requested resources by modifying the state: - `Available = Available − Request[i]` - `Allocation[i] = Allocation[i] + Request[i]` - `Need[i] = Need[i] − Request[i]` - Run the safety algorithm on this new state. If it results in a **safe state**, the allocation is completed. If it results in an **unsafe state**, Pi must wait, and the old resource-allocation state is restored.

Simple Example: Given 5 processes (P0–P4) and 3 resource types (A, B, C) with: - Available = (3, 3, 2) - Allocation and Max matrices given for each process - Need is computed as Max − Allocation

Applying the safety algorithm finds a **safe sequence** such as `<P1, P3, P4, P0, P2>`, showing the system is in a safe state (not deadlocked), so a new request is granted only if it keeps the system safe.

Advantage: Avoids deadlock without denying resource requests unnecessarily (unlike prevention). **Disadvantage:** Requires prior knowledge of maximum resource needs of each process, and the number of processes/resources must remain fixed — impractical in many real systems.

Quick Revision Summary

- **CPU scheduling** decides which ready process gets the CPU; processes alternate between **CPU bursts** and **I/O bursts**.
- **Preemptive** scheduling can take the CPU away; **Non-preemptive** cannot.
- **Scheduling criteria**: CPU utilization, throughput, turnaround time, waiting time, response time.
- **Algorithms**: FCFS (simple, convoy effect), SJF (optimal avg waiting time, non-preemptive), SRTN (preemptive SJF), Round Robin (time quantum, time-sharing), Priority Scheduling (starvation → solved by aging), Multilevel Queue (separate queues for different process types).
- **Deadlock**: processes waiting forever for resources held by each other.
- **4 necessary conditions**: Mutual Exclusion, Hold and Wait, No Preemption, Circular Wait — all must hold for deadlock.
- **Handling methods**: Prevention (deny one condition), Avoidance (Banker’s Algorithm — safe state check), Detection & Recovery.
- **Banker’s Algorithm**: uses Available, Max, Allocation, Need matrices; grants requests only if the resulting state is safe.